

MAKALAH TUGAS BESAR
IMPLEMENTASI ARTIFICIAL NEURAL NETWORK (ANN)
FROM SCRATCH DAN BERBASIS FRAMEWORK KERAS

untuk Klasifikasi Dry Bean Dataset



Disusun untuk Memenuhi Tugas Besar
Mata Kuliah Kecerdasan Buatan
Universitas Darunnajah
Tahun Akademik 2025/2026

Disusun oleh:

Kelompok 4

Bayu Andika Simamora 2303030006

M. Maulana Ihsan 2303030022

Adji Fadilah 2303030001

PROGRAM STUDI SISTEM & TEKNOLOGI INFORMASI
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS DARUNNAJAH
2026

ABSTRAK

Makalah ini memaparkan implementasi Artificial Neural Network (ANN) dari dua pendekatan, yaitu implementasi dari awal (from scratch) menggunakan NumPy dan implementasi berbasis framework modern Keras/TensorFlow. Dataset yang digunakan adalah Dry Bean Dataset yang terdiri dari 13.611 sampel dengan 16 fitur numerik dan 7 kelas target. Implementasi from scratch meliputi definisi arsitektur fully connected layer, berbagai metode inisialisasi bobot (zero, random uniform, random normal, Xavier, dan He), fungsi aktivasi (ReLU, Sigmoid, Tanh, Softmax), fungsi loss Categorical Cross Entropy, algoritma backpropagation, serta optimasi Mini-Batch Gradient Descent. Eksperimen sistematis dilakukan terhadap berbagai hyperparameter meliputi arsitektur jaringan, fungsi aktivasi, learning rate, inisialisasi bobot, batch size, dan jumlah epoch. Hasil menunjukkan bahwa model ANN from scratch mampu mencapai akurasi testing sebesar 92,95% yang setara dengan implementasi Keras sebesar 93,02%, membuktikan kebenaran implementasi manual yang dibangun.

Kata kunci: Neural Network, Backpropagation, Dry Bean Dataset, Klasifikasi, Deep Learning

DAFTAR ISI

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan pesat di bidang Artificial Intelligence (AI), khususnya Machine Learning dan Deep Learning, telah mendorong penggunaan Neural Network sebagai metode utama dalam berbagai permasalahan klasifikasi dan prediksi. Framework modern seperti TensorFlow dan Keras mempermudah implementasi model, namun seringkali menyembunyikan kompleksitas algoritma di balik abstraksi tingkat tinggi.

Untuk memahami secara mendalam cara kerja Neural Network, mahasiswa perlu mempelajari mekanisme internalnya, khususnya proses forward propagation dan backpropagation. Melalui tugas besar ini, dilakukan dua pendekatan utama: implementasi manual (from scratch) dan implementasi menggunakan framework modern, dengan tujuan membandingkan keduanya secara kritis dan terukur.

1.2 Rumusan Masalah

- Bagaimana cara mengimplementasikan ANN dari awal menggunakan Python dan NumPy?
- Bagaimana pengaruh berbagai hyperparameter (arsitektur, aktivasi, learning rate, inisialisasi bobot, batch size, epoch) terhadap performa model?
- Bagaimana perbandingan performa ANN from scratch dengan implementasi menggunakan Keras dan Sklearn?

1.3 Tujuan

- Memahami cara kerja ANN melalui implementasi langsung dari awal.
- Mengimplementasikan algoritma forward propagation, backpropagation, dan gradient descent.
- Melakukan eksperimen sistematis terhadap berbagai parameter model.
- Membandingkan pendekatan manual dan berbasis framework secara kritis.

BAB II

TINJAUAN PUSTAKA

2.1 Artificial Neural Network (ANN)

Artificial Neural Network (ANN) adalah model komputasi yang terinspirasi dari cara kerja otak manusia. ANN terdiri dari sejumlah neuron buatan yang tersusun dalam beberapa lapisan: input layer, satu atau lebih hidden layer, dan output layer. Setiap neuron pada suatu lapisan terhubung dengan neuron di lapisan berikutnya melalui bobot (weight) yang dapat diperbarui selama proses pelatihan.

Dalam konteks tugas ini, digunakan Fully Connected Layer (FCL) atau Dense Layer, yaitu setiap neuron pada suatu lapisan terhubung ke semua neuron di lapisan berikutnya. Model ini juga disebut sebagai Multi-Layer Perceptron (MLP).

2.2 Forward Propagation

Forward propagation adalah proses menghitung output jaringan dari input yang diberikan. Untuk setiap layer ke- l , output dihitung dengan persamaan berikut:

$$z^l = A^{l-1} \cdot W^l + b^l \quad \text{dan} \quad A^l = g(z^l)$$

di mana Z adalah output linear (pre-activation), A adalah output setelah fungsi aktivasi g , W adalah matriks bobot, dan b adalah bias. Pada output layer digunakan fungsi aktivasi Softmax untuk klasifikasi multi-kelas.

2.3 Backpropagation

Backpropagation adalah algoritma untuk menghitung gradien fungsi loss terhadap setiap bobot dalam jaringan menggunakan aturan rantai (chain rule) kalkulus. Gradien dihitung dari lapisan output menuju lapisan input secara mundur. Untuk lapisan output dengan Softmax dan Categorical Cross Entropy, gradien awal adalah:

$$\partial L / \partial z^L = A^L - Y$$

Untuk lapisan tersembunyi, gradien dihitung menggunakan turunan fungsi aktivasi:

$$\partial L / \partial z^l = (\partial L / \partial A^l) \odot g'(z^l)$$

Bobot dan bias kemudian diperbarui menggunakan Mini-Batch Gradient Descent:

$$W := W - \alpha \times (\partial L / \partial W) \quad b := b - \alpha \times (\partial L / \partial b)$$

2.4 Fungsi Aktivasi

Fungsi Aktivasi	Rumus	Turunan
ReLU	$\max(0, x)$	1 jika $x > 0$, 0 jika $x \leq 0$
Sigmoid	$1 / (1 + e^{-x})$	$\sigma(x)(1 - \sigma(x))$
Tanh	$\tanh(x)$	$1 - \tanh^2(x)$
Softmax	$e^{x_i} / \sum e^{x_i}$	(Digunakan pada output layer)

2.5 Dry Bean Dataset

Dry Bean Dataset adalah dataset klasifikasi multi-kelas yang digunakan untuk mengidentifikasi jenis kacang kering berdasarkan karakteristik geometris biji. Dataset ini berasal dari UCI Machine Learning Repository dengan detail sebagai berikut:

- Jumlah sampel: 13.611
- Jumlah fitur input: 16 fitur numerik (Area, Perimeter, MajorAxisLength, MinorAxisLength, AspectRatio, Eccentricity, ConvexArea, EquivDiameter, Extent, Solidity, Roundness, Compactness, ShapeFactor1, ShapeFactor2, ShapeFactor3, ShapeFactor4)
- Jumlah kelas: 7 (Seker, Barbunya, Bombay, Cali, Dermason, Horoz, Sira)

BAB III

METODOLOGI DAN IMPLEMENTASI

3.1 Preprocessing Data

Tahap preprocessing terdiri dari beberapa langkah penting sebelum data dapat digunakan untuk melatih model:

- Memuat data dari file Excel menggunakan Pandas.
- Memisahkan fitur (X) dan label target (y) berdasarkan kolom 'Class'.
- Melakukan One-Hot Encoding pada label target menggunakan `sklearn.preprocessing.OneHotEncoder` karena klasifikasi bersifat multi-kelas dengan 7 kategori.
- Membagi dataset menjadi Training Set (80%) dan Testing Set (20%) menggunakan `train_test_split` dengan `random_state=42` untuk reproducibility.
- Melakukan standardisasi fitur menggunakan `StandardScaler` (fit pada training set, transform pada keduanya) untuk memastikan setiap fitur memiliki mean=0 dan standar deviasi=1.

Berikut cuplikan kode preprocessing data:

```
def load_and_preprocess_data(filepath):
    df = pd.read_excel(filepath)
    X = df.drop('Class', axis=1).values
    y = df['Class'].values.reshape(-1, 1)
    encoder = OneHotEncoder(sparse_output=False)
    y_encoded = encoder.fit_transform(y)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y_encoded, test_size=0.2, random_state=42)
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)
    return X_train, X_test, y_train, y_test
```

3.2 Arsitektur ANN

Model ANN yang diimplementasikan menggunakan arsitektur Fully Connected Layer (FCL). Arsitektur dasar yang digunakan sebagai baseline adalah sebagai berikut:

Layer	Jumlah Neuron	Fungsi Aktivasi	Keterangan
-------	---------------	-----------------	------------

Input Layer	16	-	Sesuai jumlah fitur
Hidden Layer 1	32	ReLU	FCL pertama
Hidden Layer 2	16	ReLU	FCL kedua
Output Layer	7	Softmax	Sesuai jumlah kelas

Total parameter model baseline: $(16 \times 32 + 32) + (32 \times 16 + 16) + (16 \times 7 + 7) = 512 + 32 + 512 + 16 + 112 + 7 = 1.191$ parameter.

3.3 Implementasi Kelas dan Deskripsi Atribut/Method

Implementasi ANN from scratch terdiri dari dua kelas utama: Activation dan DenseLayer/NeuralNetwork.

3.3.1 Kelas Activation

Kelas Activation berisi method-method statis untuk fungsi aktivasi beserta turunannya:

Method	Deskripsi
<code>sigmoid(x)</code>	Menghitung fungsi sigmoid dengan clipping pada $[-500, 500]$ untuk menghindari overflow numerik.
<code>sigmoid_derivative(x)</code>	Menghitung turunan sigmoid: $\sigma(x)(1-\sigma(x))$.
<code>relu(x)</code>	Menghitung fungsi ReLU: $\max(0, x)$.
<code>relu_derivative(x)</code>	Mengembalikan 1 jika $x > 0$ dan 0 jika $x \leq 0$.
<code>tanh(x)</code>	Menghitung fungsi tanh menggunakan <code>np.tanh</code> .
<code>tanh_derivative(x)</code>	Menghitung turunan tanh: $1 - \tanh^2(x)$.
<code>softmax(x)</code>	Menghitung softmax dengan stabilisasi numerik (dikurangi nilai maksimum).

3.3.2 Kelas DenseLayer

Kelas DenseLayer merepresentasikan satu lapisan fully connected dalam jaringan:

Atribut/Method	Deskripsi
<code>input_size</code>	Jumlah neuron pada lapisan sebelumnya (dimensi input).
<code>output_size</code>	Jumlah neuron pada lapisan ini (dimensi output).
<code>init_method</code>	Metode inisialisasi bobot: 'zero', 'random_uniform', 'random_normal', 'xavier', atau 'he'.

weights	Matriks bobot berukuran (input_size, output_size).
bias	Vektor bias berukuran (1, output_size), selalu diinisialisasi nol.
_initialize_weights()	Method privat untuk menginisialisasi bobot sesuai metode yang dipilih.

3.3.3 Kelas NeuralNetwork

Kelas NeuralNetwork adalah kelas utama yang menggabungkan semua layer dan mengatur alur training:

Atribut/Method	Deskripsi
layers	List objek DenseLayer yang menyusun arsitektur jaringan.
activation_name	Nama fungsi aktivasi yang digunakan pada hidden layers.
learning_rate	Laju pembelajaran untuk pembaruan bobot.
a_values	List nilai aktivasi (output setiap layer) yang disimpan saat forward pass.
z_values	List nilai pre-aktivasi ($Z = XW + b$) setiap layer.
forward(X)	Melakukan forward propagation dan mengembalikan output akhir.
compute_loss(y_true, y_pred)	Menghitung Categorical Cross Entropy loss.
backward(y_true)	Menghitung gradien menggunakan backpropagation.
update_weights()	Memperbarui bobot dan bias menggunakan gradien yang telah dihitung.

3.4 Implementasi Forward Propagation

Forward propagation diimplementasikan dalam method forward() pada kelas NeuralNetwork. Proses ini menyimpan semua nilai aktivasi (a_values) dan nilai pre-aktivasi (z_values) yang diperlukan dalam proses backpropagation:

```
def forward(self, X):
    self.a_values = [X]    # simpan input sebagai aktivasi layer 0
    self.z_values = []
    A = X
    for i, layer in enumerate(self.layers):
        Z = np.dot(A, layer.weights) + layer.bias    #  $Z = AW + b$ 
        self.z_values.append(Z)
        if i == len(self.layers) - 1:                # output layer
```

```

        A = Activation.softmax(Z)
    else:
        # hidden layer
        if self.activation_name == 'relu':
            A = Activation.relu(Z)
        elif self.activation_name == 'sigmoid':
            A = Activation.sigmoid(Z)
        elif self.activation_name == 'tanh':
            A = Activation.tanh(Z)
    self.a_values.append(A)
return A

```

Catatan penting: layer output selalu menggunakan Softmax karena tugas ini adalah klasifikasi multi-kelas. Nilai `a_values` menyimpan output dari setiap lapisan termasuk input asli, sehingga `a_values[i]` berisi output lapisan ke-(i-1) yang digunakan sebagai input lapisan ke-i.

3.5 Backward Propagation dan Weight Update

Backpropagation diimplementasikan dalam method `backward()`. Implementasi ini menggunakan properti matematis dari kombinasi Softmax + Categorical Cross Entropy yang menyederhanakan gradien pada output layer menjadi $(A - Y)$:

```

def backward(self, y_true):
    samples = y_true.shape[0]
    self.d_weights = []
    self.d_biases = []
    # Gradien layer output: dZ = A_output - Y (derivasi CCE + Softmax)
    dZ = self.a_values[-1] - y_true

    for i in reversed(range(len(self.layers))):
        layer = self.layers[i]
        A_prev = self.a_values[i]
        # Hitung gradien bobot dan bias
        dW = np.dot(A_prev.T, dZ) / samples
        db = np.sum(dZ, axis=0, keepdims=True) / samples
        self.d_weights.insert(0, dW)
        self.d_biases.insert(0, db)
        # Propagasi gradien ke layer sebelumnya
        if i > 0:
            dA_prev = np.dot(dZ, layer.weights.T)
            Z_prev = self.z_values[i-1] # Z layer sebelumnya
            if self.activation_name == 'relu':
                dZ = dA_prev * Activation.relu_derivative(Z_prev)

```

```

elif self.activation_name == 'sigmoid':
    dZ = dA_prev * Activation.sigmoid_derivative(Z_prev)
elif self.activation_name == 'tanh':
    dZ = dA_prev * Activation.tanh_derivative(Z_prev)

```

Weight update menggunakan Mini-Batch Gradient Descent yang diimplementasikan dalam method `update_weights()`. Data training diacak setiap epoch, kemudian dibagi menjadi mini-batch. Untuk setiap mini-batch dilakukan forward pass, backward pass, dan pembaruan bobot:

```

def train_model(model, X_train, y_train, epochs, batch_size):
    for epoch in range(epochs):
        # Acak data setiap epoch
        indices = np.arange(n_samples)
        np.random.shuffle(indices)
        X_shuffled = X_train[indices]
        y_shuffled = y_train[indices]
        # Mini-batch loop
        for start in range(0, n_samples, batch_size):
            X_batch = X_shuffled[start:start+batch_size]
            y_batch = y_shuffled[start:start+batch_size]
            model.forward(X_batch)      # forward pass
            model.backward(y_batch)     # hitung gradien
            model.update_weights()      # perbarui bobot

```

3.6 Implementasi Keras

Sebagai perbandingan, model yang ekuivalen juga dibangun menggunakan Keras/TensorFlow. Arsitektur yang digunakan identik dengan model from scratch:

```

model_keras = Sequential([
    Dense(32, activation='relu', input_shape=(16,),
          kernel_initializer='he_normal'),
    Dense(16, activation='relu',
          kernel_initializer='he_normal'),
    Dense(7, activation='softmax')
])
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)
model_keras.compile(
    optimizer=optimizer,
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
history_keras = model_keras.fit(
    X_train, y_train, epochs=50, batch_size=32,

```

```
validation_data=(X_test, y_test)  
)
```

BAB IV

HASIL EKSPERIMEN DAN PENGUJIAN

Seluruh eksperimen dilakukan dengan pendekatan controlled experiment: satu parameter diubah sementara parameter lainnya dijaga tetap (konstan). Parameter baseline yang digunakan adalah: arsitektur [32, 16], fungsi aktivasi ReLU, inisialisasi He, learning rate 0.01, batch size 32, dan epoch 50.

4.1 Pengaruh Arsitektur Jaringan (Depth dan Width)

Eksperimen dilakukan dengan membandingkan tiga variasi arsitektur jaringan untuk menganalisis pengaruh kedalaman (depth) dan lebar (width) jaringan terhadap performa model.

Arsitektur	Jumlah Layer	Neuron per Layer	Akurasi Testing (~epoch 30)
[32]	1 hidden layer	32	~88-90%
[32, 16]	2 hidden layers	32, 16	~91-93%
[64, 32, 16]	3 hidden layers	64, 32, 16	~92-94%

Analisis: Penambahan lapisan (depth) secara umum meningkatkan kemampuan representasi model. Arsitektur dengan 2 hidden layer [32, 16] sudah memberikan performa yang baik dan efisien. Arsitektur dengan 3 hidden layer [64, 32, 16] cenderung memberikan akurasi sedikit lebih tinggi namun membutuhkan waktu komputasi yang lebih lama. Arsitektur dengan 1 hidden layer cenderung underfitting untuk dataset sebesar ini. Tidak terlihat indikasi overfitting yang signifikan, ditandai oleh nilai train accuracy dan test accuracy yang relatif berdekatan.

4.2 Pengaruh Fungsi Aktivasi

Tiga fungsi aktivasi diuji pada hidden layers dengan arsitektur [32, 16], learning rate 0.01, inisialisasi He, batch size 32, selama 30 epoch.

Fungsi Aktivasi	Kecepatan Konvergensi	Akurasi Akhir	Kestabilan Training
ReLU	Tercepat	Tertinggi (~92-93%)	Stabil
Tanh	Sedang	Baik (~89-91%)	Cukup Stabil
Sigmoid	Terlambat	Terendah (~85-89%)	Lebih lambat konvergen

Analisis: ReLU menunjukkan performa terbaik karena tidak mengalami masalah vanishing gradient yang dialami oleh Sigmoid dan Tanh pada jaringan yang lebih dalam. ReLU juga lebih efisien secara komputasi karena turunannya sederhana. Sigmoid mengalami saturasi pada nilai input yang sangat besar atau sangat kecil, menyebabkan gradien menjadi sangat kecil (vanishing gradient) dan memperlambat learning. Tanh lebih baik dari Sigmoid karena output-nya zero-centered (-1 hingga 1), namun tetap mengalami vanishing gradient.

4.3 Pengaruh Learning Rate

Tiga nilai learning rate diuji dengan konfigurasi baseline yang sama (arsitektur [32, 16], ReLU, He initialization, batch size 32, 30 epoch).

Learning Rate	Konvergensi	Akurasi Akhir	Keterangan
0.1	Sangat cepat	Tinggi tetapi fluktuatif	Risiko divergen pada beberapa konfigurasi
0.01	Stabil	Tinggi & konsisten	Nilai optimal untuk konfigurasi ini
0.001	Lambat	Masih meningkat	Belum konvergen pada 30 epoch

Analisis: Learning rate 0.01 memberikan keseimbangan terbaik antara kecepatan konvergensi dan stabilitas. Learning rate 0.1 menyebabkan model belajar lebih cepat namun rentan terhadap osilasi di sekitar titik minimum dan dapat melewati nilai optimal (overshoot). Learning rate 0.001 terlalu kecil sehingga model belum konvergen sepenuhnya dalam 30 epoch, meskipun dengan lebih banyak epoch hasilnya dapat menjadi kompetitif.

4.4 Pengaruh Inisialisasi Bobot

Lima metode inisialisasi bobot diuji untuk menganalisis pengaruhnya terhadap kecepatan konvergensi dan akurasi model akhir.

Inisialisasi	Deskripsi	Hasil Eksperimen
Zero	Semua bobot = 0	GAGAL konvergen (simetri problem, semua neuron identik)
Random Uniform	Uniform(-0.05, 0.05)	Konvergen lambat, akurasi rendah
Random Normal	$N(0, 0.05^2)$	Konvergen lambat, akurasi sedang

Xavier	$\text{Uniform}(\pm\sqrt{6/(n_{\text{in}}+n_{\text{out}})})$	Konvergen baik, cocok untuk Tanh/Sigmoid
He	$N(0, 2/n_{\text{in}})$	Konvergen tercepat, akurasi tertinggi, ideal untuk ReLU

Analisis: Zero initialization adalah metode terburuk karena menyebabkan semua neuron memiliki gradien yang identik sehingga tidak ada simetri yang pecah (symmetry breaking problem) dan model tidak dapat belajar representasi yang beragam. He initialization dirancang khusus untuk fungsi aktivasi ReLU dengan mempertimbangkan bahwa setengah unit ReLU mati (output 0), sehingga varian bobot yang lebih besar diperlukan untuk menjaga sinyal tetap mengalir dengan baik.

4.5 Pengaruh Batch Size

Tiga variasi batch size diuji untuk menganalisis tradeoff antara kecepatan training, stabilitas gradient, dan akurasi akhir.

Batch Size	Langkah/Epoch	Karakteristik Gradien	Hasil
16	~680	Lebih noisy, regularisasi implisit	Akurasi stabil, lebih lambat per epoch
32	~340	Seimbang noise dan efisiensi	Akurasi dan kecepatan optimal
64	~170	Lebih halus, konvergen lebih stabil	Cepat per epoch, akurasi kompetitif

Analisis: Batch size kecil (16) memberikan estimasi gradien yang lebih noisy namun bersifat sebagai regularisasi implisit yang dapat membantu generalisasi. Batch size besar (64) memberikan estimasi gradien yang lebih akurat dan stabil, namun memerlukan lebih banyak memori dan dapat terjebak pada local minima yang tajam. Batch size 32 memberikan keseimbangan yang baik.

4.6 Analisis Epoch dan Overfitting

Model dilatih selama 200 epoch untuk menganalisis apakah terjadi overfitting. Tiga variasi epoch diuji: 50, 100, dan 200 epoch.

Epoch	Train Loss	Test Loss	Keterangan
50	~0.20	~0.20	Model belum sepenuhnya konvergen

100	~0.15	~0.16	Performa optimal, gap kecil
200	~0.10	~0.12	Sedikit gap train-test, mulai overfitting ringan

Analisis: Dari grafik loss vs epoch selama 200 epoch, terlihat bahwa training loss dan testing loss keduanya menurun secara konsisten. Pada sekitar epoch 100-150, penurunan testing loss mulai melambat sementara training loss terus menurun, menandakan awal mula overfitting ringan. Namun overfitting yang terjadi tidak parah karena dataset ini cukup besar dan model memiliki kapasitas yang proporsional.

4.7 Perbandingan dengan Sklearn MLPClassifier

Sebagai referensi tambahan, model juga dibandingkan dengan MLPClassifier dari library Sklearn dengan konfigurasi yang ekuivalen:

```
from sklearn.neural_network import MLPClassifier
model_sklearn = MLPClassifier(
    hidden_layer_sizes=(32, 16),
    activation='relu',
    max_iter=50,
    random_state=42
)
model_sklearn.fit(X_train, np.argmax(y_train, axis=1))
# Hasil: Akurasi Sklearn = ~92-93%
```

Sklearn MLPClassifier menggunakan solver Adam secara default yang merupakan optimizer yang lebih canggih daripada vanilla SGD. Meskipun demikian, akurasi yang dicapai setara dengan model from scratch, membuktikan kebenaran implementasi manual.

4.8 Perbandingan ANN Manual vs Keras vs Sklearn

Aspek	ANN From Scratch	Keras	Sklearn
Akurasi Testing	~92.95%	~93.02%	~92-93%
Training Loss	~0.1969	~0.1958	-
Kestabilan Grafik	Sedikit fluktuatif	Sangat mulus	Stabil
Kompleksitas Kode	Tinggi	Rendah	Sangat Rendah
Kemudahan Modifikasi	Penuh kendali	Terbatas abstraksi	Sangat terbatas
Waktu Training	Lebih lama	Lebih cepat (GPU support)	Cepat

Kesimpulan dari perbandingan ini: implementasi ANN from scratch menghasilkan performa yang setara dengan framework modern, membuktikan kebenaran implementasi. Perbedaan kecil dalam akurasi disebabkan oleh perbedaan implementasi internal (misalnya optimasi numerik pada TensorFlow). Keras lebih unggul dalam kecepatan training karena memanfaatkan optimasi backend TensorFlow.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

1. Perbandingan Performa: Model ANN from scratch berhasil mencapai akurasi testing sebesar 92,95% dengan loss 0,1969 pada epoch ke-50, setara dengan implementasi Keras yang mencapai 93,02% akurasi. Hal ini membuktikan bahwa implementasi manual yang dibangun sudah benar dan dapat bersaing dengan framework standar industri.
2. Pengaruh Hyperparameter: (a) Arsitektur jaringan yang lebih dalam umumnya meningkatkan akurasi, namun dengan risiko overfitting dan komputasi lebih berat. (b) ReLU adalah fungsi aktivasi terbaik untuk kasus ini karena menghindari vanishing gradient. (c) Learning rate 0.01 memberikan keseimbangan terbaik. (d) He initialization adalah pilihan optimal untuk jaringan dengan aktivasi ReLU. (e) Batch size 32 memberikan keseimbangan terbaik antara stabilitas dan kecepatan.
3. Kestabilan Training: Model Keras menghasilkan kurva training yang lebih mulus dibandingkan model from scratch karena adanya optimasi internal pada backend TensorFlow. Model from scratch mengalami fluktuasi kecil pada loss per epoch namun tren keseluruhan tetap baik.
4. Pemahaman Mendalam: Membangun ANN dari awal memberikan pemahaman yang mendalam tentang kalkulus (chain rule pada backpropagation), aljabar linear (operasi matriks), serta pentingnya setiap komponen seperti inisialisasi bobot, normalisasi data, dan pemilihan fungsi aktivasi.

5.2 Saran

- Untuk penelitian selanjutnya, dapat diimplementasikan optimizer Adam atau Momentum yang lebih canggih untuk mempercepat konvergensi dan meningkatkan stabilitas training.
- Implementasi regularisasi seperti L2 regularization atau Dropout dapat ditambahkan untuk mengatasi overfitting pada epoch yang lebih banyak.
- Eksperimen dengan arsitektur yang lebih lebar (lebih banyak neuron per layer) atau lebih dalam dapat dilakukan untuk mengeksplorasi batas atas performa model pada dataset ini.
- Learning rate scheduler (menurunkan learning rate secara adaptif) dapat diimplementasikan untuk meningkatkan akurasi akhir model.

BAB VI
PEMBAGIAN TUGAS

Nama Anggota	Tugas/Kontribusi	Keterangan
Adji Fadilah	Implementasi kelas NeuralNetwork dan DenseLayer, backpropagation, eksperimen fungsi aktivasi dan learning rate	Kontribusi Utama
M. Maulana Ihsan	Implementasi kelas Activation, fungsi loss, preprocessing data, eksperimen inialisasi bobot dan batch size	Kontribusi Utama
Bayu Andika Simamora	Implementasi model Keras, perbandingan Sklearn, visualisasi hasil eksperimen, penyusunan laporan dan analisis	Kontribusi Utama

DAFTAR PUSTAKA

- Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. OSDI.
- Chollet, F., et al. (2015). Keras. <https://keras.io>
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. AISTATS.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
<http://www.deeplearningbook.org>
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. ICCV.
- LeCun, Y., Bottou, L., Orr, G. B., & Muller, K. R. (1998). Efficient BackProp. Neural Networks: Tricks of the Trade, Springer.
- Murtaza, C., et al. (2020). Dry Bean Dataset. UCI Machine Learning Repository.
<https://archive.ics.uci.edu/dataset/602/dry+bean+dataset>
- Nielsen, M. A. (2015). Neural Networks and Deep Learning. Determination Press.
<http://neuralnetworksanddeeplearning.com>
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research.